Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2025

Introduction to Software Engineering

Software Design Document (SDD)

Project: Clinexa Medical System

Version 1.0

Team Number: 26

Email: s-ahmed.ragab@zewailcity.edu.eg

Team Members

Name	**ID**	**Email**
Ahmed Ragab	202400437	s-ahmed.ragab@zewailcity.edu.eg
Mosab Kamal	202402297	s-mosab.gabal@zewailcity.edu.eg
Mohamed Maher	202301798	s-mohammed.kafafy@zewailcity.edu.eg
Abdelrhman Khaled	202400161	s-abdelrhman.abdelhamid@zewailcity.edu.eg

Contents

1 Introduction **3**

1.1 Purpose of the Document . . . 3

1.2 Scope of the Design Phase . . . 3

1.3 Intended Audience . . . 4

1.4 Overview of the Contents . . . 4

2 System Overview **5**

2.1 Brief Description of the System . . . 5

2.2 Key Design Goals and Constraints . . . 5

2.2.1 Design Goals . . . 5

2.2.2 Constraints . . . 5

3 Architectural Design **6**

3.1 System Architecture Diagram . . . 6

3.2 Discussion of Architectural Style and Components . . . 7

3.2.1 Monolithic Architecture . . . 7

3.3 Technology Stack and Tools . . . 8

3.3.1 Backend Technologies . . . 8

3.3.2 Frontend Technologies . . . 8

3.3.3 Data Storage . . . 8

3.3.4 External Services . . . 8

4 Detailed Design **9**

4.1 Model–View–Controller (MVC) Design Pattern . . . 9

4.1.1 Description of MVC Pattern . . . 9

4.1.2 Mapping of Project Components to MVC . . . 9

4.1.3 Interaction Between Components . . . 11

4.2 UML Diagrams . . . 14

4.2.1 Class Diagram . . . 14

4.2.2 Sequence Diagrams . . . 15

4.3 UI/UX Design . . . 18

4.3.1 Wireframes / Mockups 18
4.4 Data Design 23
4.4.1 File Structure / Data Storage Model 23
4.4.2 File Schemas 23

5 Conclusion **26**
5.1 Summary of Design Decisions 26
5.2 Design Phase Outputs 26
5.3 Alignment with Requirements 26
5.4 Next Steps 27

1 Introduction

1.1 Purpose of the Document

This Software Design Document (SDD) defines the architectural and detailed design of the Clinexa Medical System based on the Software Requirements Specification (SRS). The document serves as a comprehensive blueprint that translates functional and non-functional requirements into a structured design that can be implemented by the development team.

The SDD provides:

- A clear explanation of the system's architectural structure and design patterns
- Detailed descriptions of system components and their interactions
- Visual representations through UML diagrams and UI mockups
- Data storage design and file structure specifications
- Technology stack and implementation guidelines

1.2 Scope of the Design Phase

This design document covers the complete system design for the Clinexa Medical System, including:

- **Architectural Design:** High-level system architecture using a monolithic architectural style, component identification, and interaction patterns.
- **Design Patterns:** Implementation of the Model-View-Controller (MVC) design pattern.
- **Module Design:** Detailed design of User Management, Authentication, Patient Management, Doctor Management, Appointment Management, Medical Records, Notifications, Reports, and Search functionality.
- **Data Storage Design:** File-based data storage structure using text files.
- **User Interface Design:** Wireframes and mockups for key user interfaces.

- **Component Interactions:** UML diagrams illustrating system behavior.

1.3 Intended Audience

This Software Design Document is intended for:

- **Software Developers:** Primary audience who will implement the system.
- **Quality Assurance Testers:** For understanding system structure and creating test plans.
- **Project Stakeholders:** Project managers and healthcare administrators.
- **Course Instructors and Evaluators:** For assessing design quality.
- **Future Maintainers:** Developers maintaining or extending the system.

1.4 Overview of the Contents

- **Section 1 – Introduction:** Purpose, scope, audience, and document overview.
- **Section 2 – System Overview:** High-level summary, design goals, and constraints.
- **Section 3 – Architectural Design:** System architecture and technology stack.
- **Section 4 – Detailed Design:** MVC pattern, UML diagrams, UI/UX design, and data design.
- **Section 5 – Conclusion:** Summary of design decisions and next steps.

2 System Overview

2.1 Brief Description of the System

Clinexa is a web-based healthcare management system designed to streamline interaction between healthcare providers and patients. The platform replaces traditional paper-based processes with a digital solution accessible through web browsers.

Patients can create accounts, search for doctors by specialization, book appointments, view medical history, receive automated email reminders, and submit feedback. Doctors can manage availability schedules, view appointments, access patient histories, create medical records, generate reports, and respond to feedback.

2.2 Key Design Goals and Constraints

2.2.1 Design Goals

- **Usability:** Intuitive interface requiring minimal training, accessible through modern web browsers.
- **Efficiency:** Streamlined processes for appointment scheduling and medical record retrieval.
- **Reliability:** 99.9% uptime with recovery time under 5 minutes.
- **Maintainability:** Modular design with clear separation of concerns through MVC pattern.
- **Security:** Robust authentication with password hashing and role-based access control.

2.2.2 Constraints

- **Technology Stack:** Flask backend, HTML/CSS frontend, file-based storage, UML 2.5 notation.
- **Operating Environment:** Chrome and Firefox browsers, cloud server hosting.
- **External Dependencies:** Email gateway API for notifications.

3 Architectural Design

3.1 System Architecture Diagram

The Clinexa Medical System follows a three-tier monolithic architecture. Figure 1 illustrates the overall system architecture.

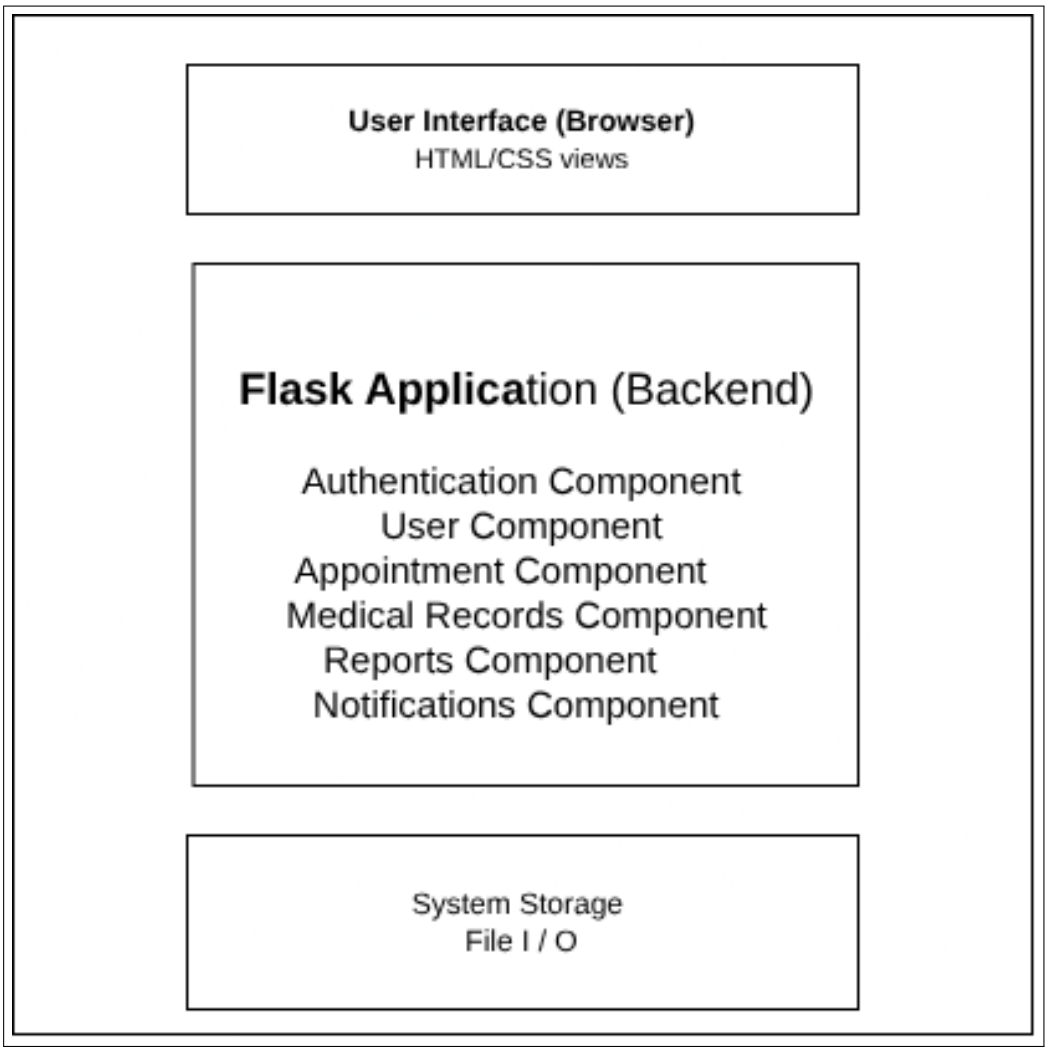


Figure 1: Clinexa System Architecture - Three-Tier Monolithic Design

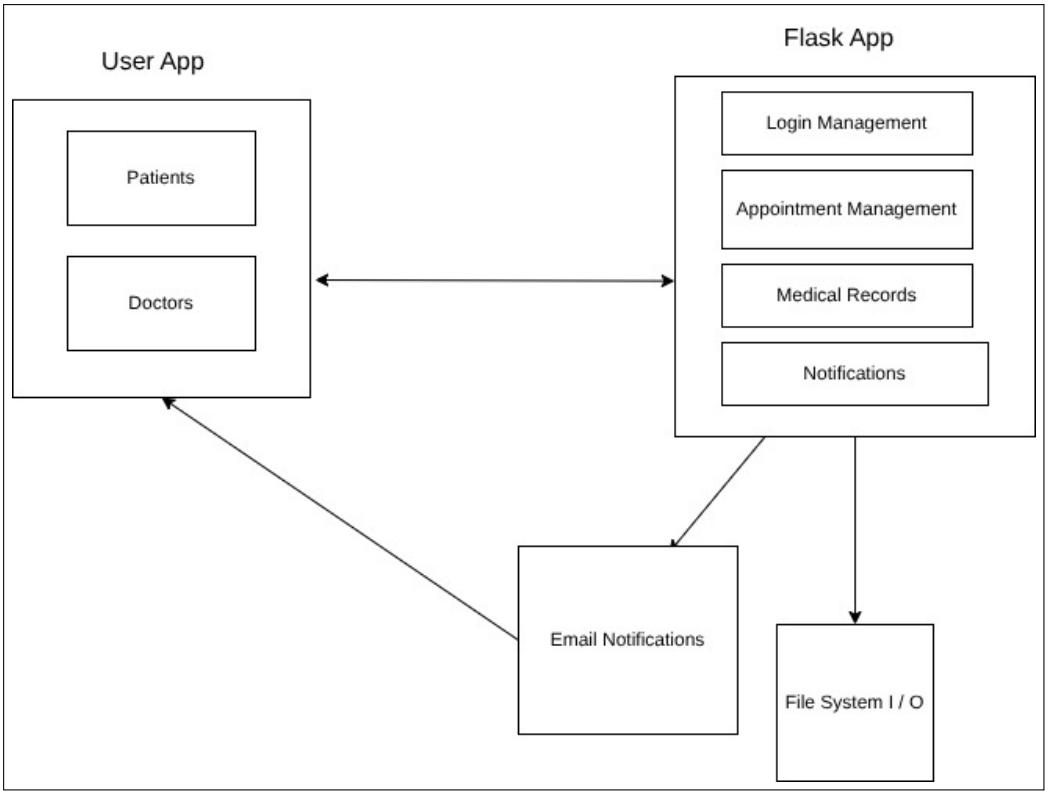


Figure 2: Detailed Architecture View - Component breakdown

The architecture consists of:

1. **Presentation Layer:** Web Browser Interface (HTML/CSS)
2. **Business Logic Layer:** Flask Web Application Server
3. **Data Access Layer:** File system management

3.2 Discussion of Architectural Style and Components

3.2.1 Monolithic Architecture

Clinexa adopts a monolithic architectural style where all components are integrated into a single application.

Rationale:

- Well-defined project scope manageable as a single application
- Simplified development for student team
- Flask optimized for monolithic applications
- Straightforward deployment to cloud server

3.3 Technology Stack and Tools

3.3.1 Backend Technologies

- **Programming Language:** Python 3.x
- **Web Framework:** Flask 2.x with Jinja2 templates

3.3.2 Frontend Technologies

- **Markup Language:** HTML5
- **Styling:** CSS3

3.3.3 Data Storage

- **Storage Mechanism:** File-based text storage with pipe-separated values (PSV)

3.3.4 External Services

- **Email Service:** SMTP-compatible email service for appointment notifications
- **Hosting:** Cloud hosting platform (Heroku, PythonAnywhere)

4 Detailed Design

4.1 Model–View–Controller (MVC) Design Pattern

4.1.1 Description of MVC Pattern

The Model-View-Controller (MVC) is an architectural design pattern that separates an application into three interconnected components.

Components:

- **Model:** Data layer and business logic
- **View:** User interface presentation
- **Controller:** Request handling and flow control

Why MVC Was Chosen:

1. Separation of concerns for maintainability
2. Enables parallel development
3. Improves testability
4. Framework compatibility with Flask
5. Supports future scalability

4.1.2 Mapping of Project Components to MVC

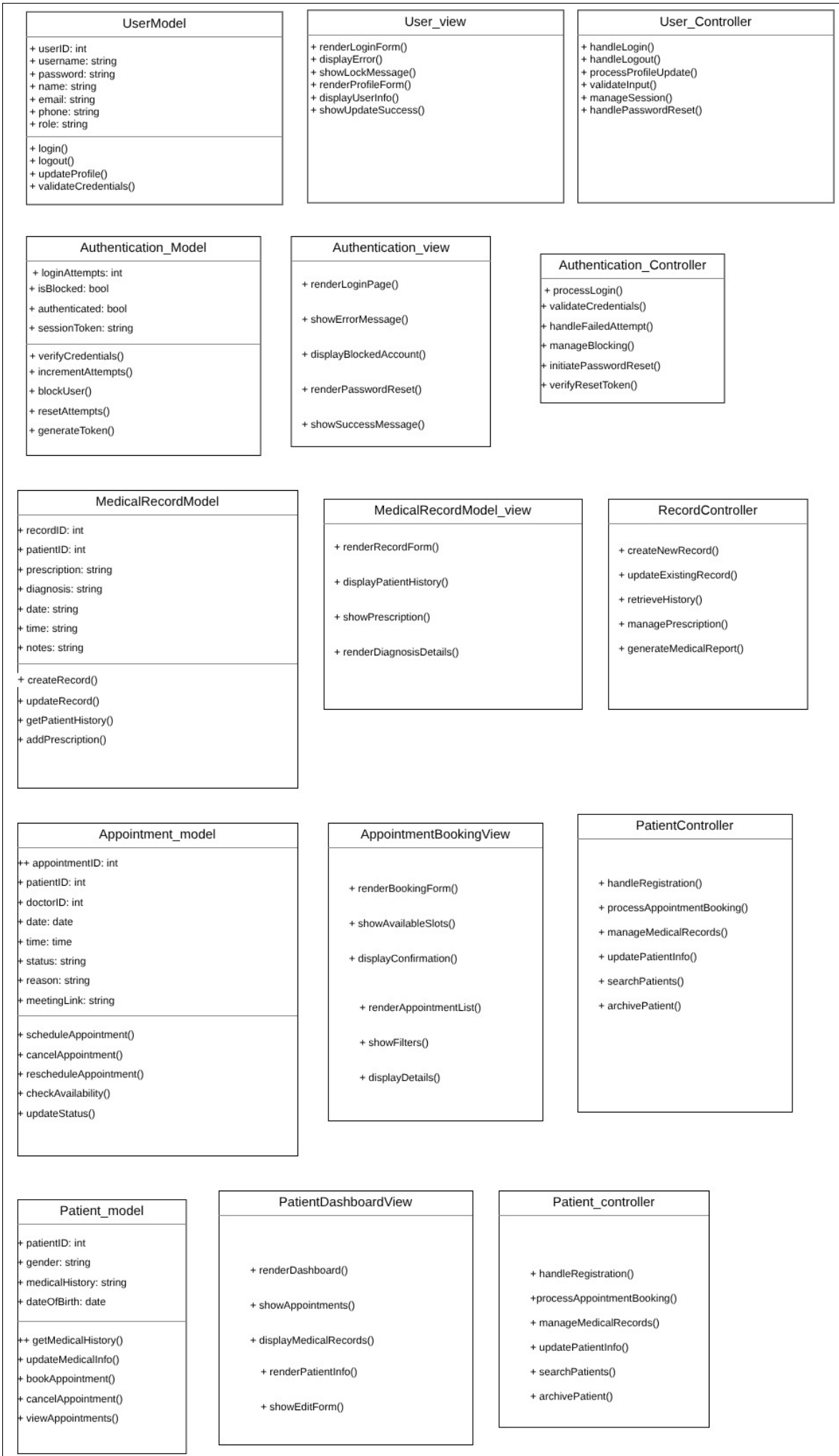
Table 1 provides a comprehensive mapping of Clinexa components to the MVC pattern.

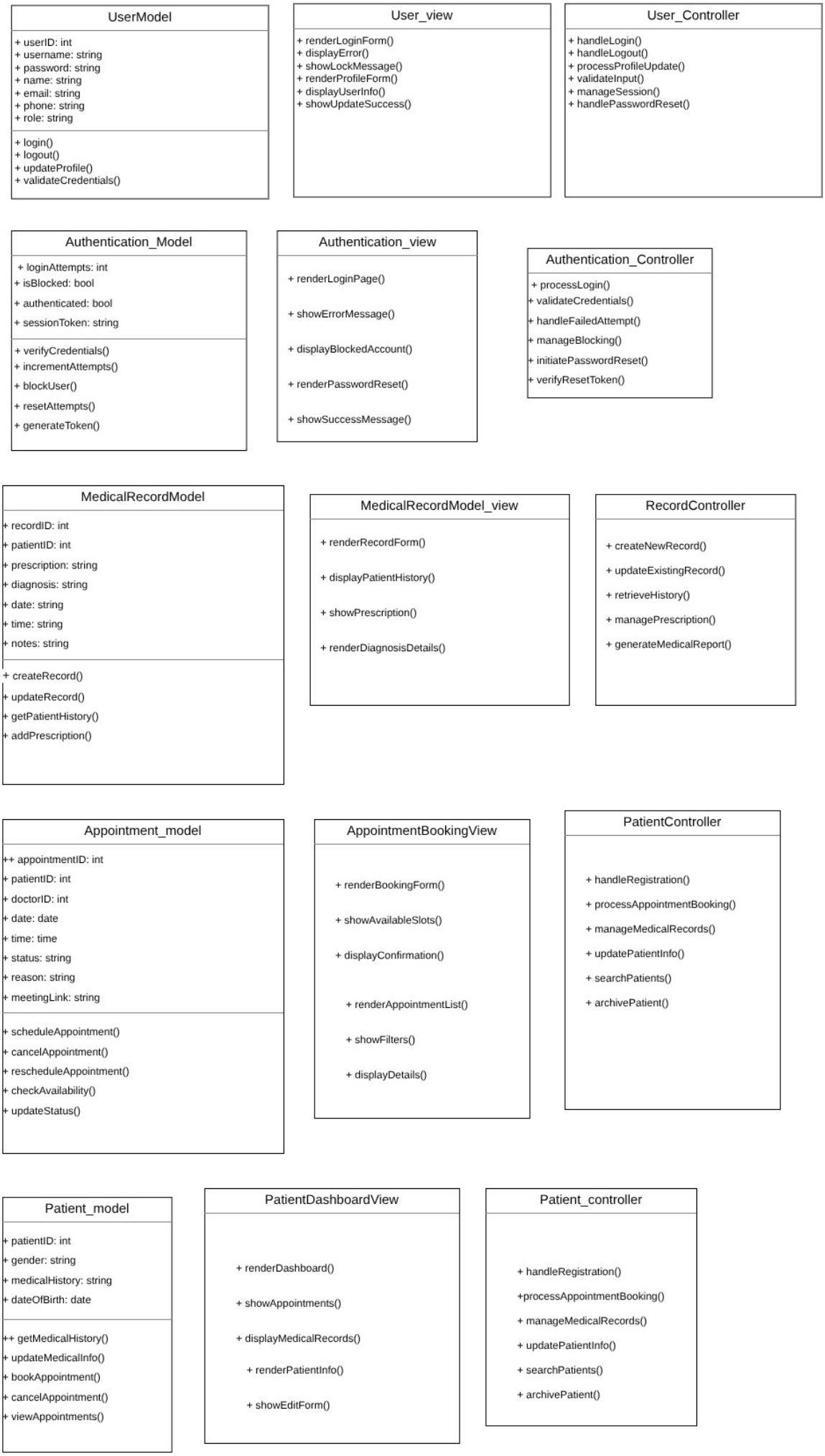
Table 1: Mapping of Clinexa Components to MVC Pattern

Component	**Model**	**View**	**Controller**
User Management	User class (userID, username, password, name, email, phone, role) with login(), logout(), updateProfile() methods	UserLoginView, UserProfileView with forms	UserController handles login, logout, profile updates, sessions
Authentication	Authentication class tracking login attempts, blocking status	AuthenticationView with login page, error messages	AuthController validates credentials, manages blocking
Patient Management	Patient class with medical history, demographics	PatientDashboardView, PatientProfileView	PatientController manages registration, appointments
Doctor Management	Doctor class with specialization, availability	DoctorDashboardView, DoctorScheduleView	DoctorController manages schedules, records access
Appointment Management	Appointment class (date, time, status, IDs)	AppointmentBookingView, AppointmentListView	AppointmentController processes bookings, checks availability
Medical Records	AppointmentRecord class (diagnoses, prescriptions)	MedicalRecordView with forms	RecordController creates/updates records
Notifications	Notification class (message, recipient, status)	NotificationView with lists	NotificationController sends reminders
Report Generation	Report class (type, parameters, data)	ReportView with filters, charts	ReportController processes requests, formats reports
Search	Search class (query, filters, results)	SearchView with search bar	SearchController processes queries, applies filters

4.1.3 Interaction Between Components

Figure 3 illustrates how Model, View, and Controller interact during typical operations.


UserModel
+ userID: int
+ username: string
+ password: string
+ name: string
+ email: string
+ phone: string
+ role: string
+ login()
+ logout()
+ updateProfile()
+ validateCredentials()
User_view
+ renderLoginForm()
+ displayError()
+ showLockMessage()
+ renderProfileForm()
+ displayUserInfo()
+ showUpdateSuccess()
User_Controller
+ handleLogin()
+ handleLogout()
+ processProfileUpdate()
+ validateInput()
+ manageSession()
+ handlePasswordReset()
Authentication_Model
+ loginAttempts: int
+ isBlocked: bool
+ authenticated: bool
+ sessionToken: string
+ verifyCredentials()
+ incrementAttempts()
+ blockUser()
+ resetAttempts()
+ generateToken()
Authentication_view
+ renderLoginPage()
+ showErrorMessage()
+ displayBlockedAccount()
+ renderPasswordReset()
+ showSuccessMessage()
Authentication_Controller
+ processLogin()
+ validateCredentials()
+ handleFailedAttempt()
+ manageBlocking()
+ initiatePasswordReset()
+ verifyResetToken()
MedicalRecordModel
+ recordID: int
+ patientID: int
+ prescription: string
+ diagnosis: string
+ date: string
+ time: string
+ notes: string
+ createRecord()
+ updateRecord()
+ getPatientHistory()
+ addPrescription()
MedicalRecordModel_view
+ renderRecordForm()
+ displayPatientHistory()
+ showPrescription()
+ renderDiagnosisDetails()
RecordController
+ createNewRecord()
+ updateExistingRecord()
+ retrieveHistory()
+ managePrescription()
+ generateMedicalReport()
Appointment_model
++ appointmentID: int
+ patientID: int
+ doctorID: int
+ date: date
+ time: time
+ status: string
+ reason: string
+ meetingLink: string
+ scheduleAppointment()
+ cancelAppointment()
+ rescheduleAppointment()
+ checkAvailability()
+ updateStatus()
AppointmentBookingView
+ renderBookingForm()
+ showAvailableSlots()
+ displayConfirmation()
+ renderAppointmentList()
+ showFilters()
+ displayDetails()
PatientController
+ handleRegistration()
+ processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()
Patient_model
+ patientID: int
+ gender: string
+ medicalHistory: string
+ dateOfBirth: date
++ getMedicalHistory()
+ updateMedicalInfo()
+ bookAppointment()
+ cancelAppointment()
+ viewAppointments()
PatientDashboardView
+ renderDashboard()
+ showAppointments()
+ displayMedicalRecords()
+ renderPatientInfo()
+ showEditForm()
Patient_controller
+ handleRegistration()
+processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()


UserModel
+ userID: int
+ username: string
+ password: string
+ name: string
+ email: string
+ phone: string
+ role: string
+ login()
+ logout()
+ updateProfile()
+ validateCredentials()
User_view
+ renderLoginForm()
+ displayError()
+ showLockMessage()
+ renderProfileForm()
+ displayUserInfo()
+ showUpdateSuccess()
User_Controller
+ handleLogin()
+ handleLogout()
+ processProfileUpdate()
+ validateInput()
+ manageSession()
+ handlePasswordReset()
Authentication_Model
+ loginAttempts: int
+ isBlocked: bool
+ authenticated: bool
+ sessionToken: string
+ verifyCredentials()
+ incrementAttempts()
+ blockUser()
+ resetAttempts()
+ generateToken()
Authentication_view
+ renderLoginPage()
+ showErrorMessage()
+ displayBlockedAccount()
+ renderPasswordReset()
+ showSuccessMessage()
Authentication_Controller
+ processLogin()
+ validateCredentials()
+ handleFailedAttempt()
+ manageBlocking()
+ initiatePasswordReset()
+ verifyResetToken()
MedicalRecordModel
+ recordID: int
+ patientID: int
+ prescription: string
+ diagnosis: string
+ date: string
+ time: string
+ notes: string
+ createRecord()
+ updateRecord()
+ getPatientHistory()
+ addPrescription()
MedicalRecordModel_view
+ renderRecordForm()
+ displayPatientHistory()
+ showPrescription()
+ renderDiagnosisDetails()
RecordController
+ createNewRecord()
+ updateExistingRecord()
+ retrieveHistory()
+ managePrescription()
+ generateMedicalReport()
Appointment_model
++ appointmentID: int
+ patientID: int
+ doctorID: int
+ date: date
+ time: time
+ status: string
+ reason: string
+ meetingLink: string
+ scheduleAppointment()
+ cancelAppointment()
+ rescheduleAppointment()
+ checkAvailability()
+ updateStatus()
AppointmentBookingView
+ renderBookingForm()
+ showAvailableSlots()
+ displayConfirmation()
+ renderAppointmentList()
+ showFilters()
+ displayDetails()
PatientController
+ handleRegistration()
+ processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()
Patient_model
+ patientID: int
+ gender: string
+ medicalHistory: string
+ dateOfBirth: date
++ getMedicalHistory()
+ updateMedicalInfo()
+ bookAppointment()
+ cancelAppointment()
+ viewAppointments()
PatientDashboardView
+ renderDashboard()
+ showAppointments()
+ displayMedicalRecords()
+ renderPatientInfo()
+ showEditForm()
Patient_controller
+ handleRegistration()
+processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()

4.2 UML Diagrams

4.2.1 Class Diagram

Figure 5 presents the complete class structure of the Clinexa system, showing all major classes, their attributes, methods, and relationships.

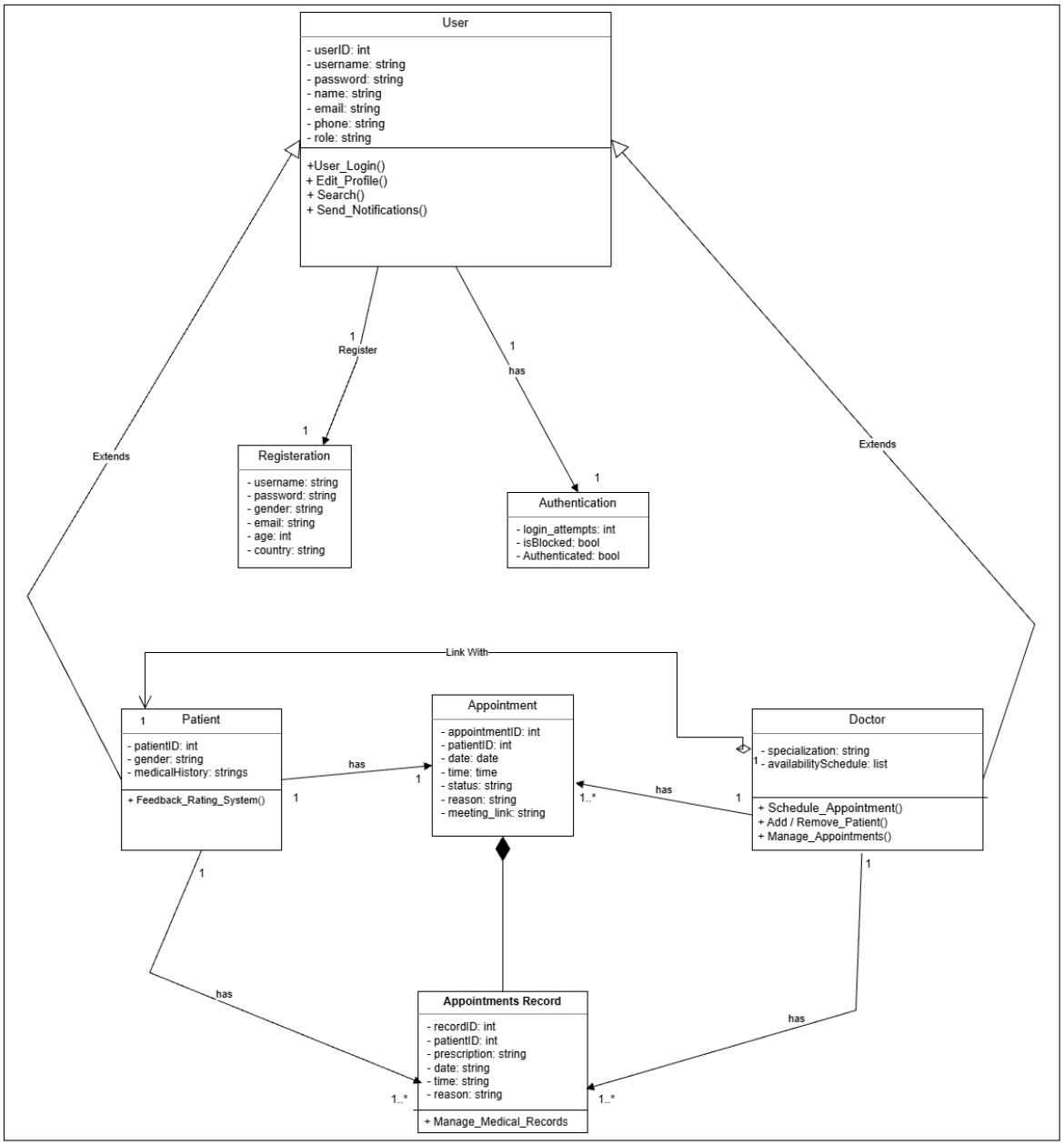


Figure 5: Class Diagram - Complete class structure showing User, Patient, Doctor, Appointment, MedicalRecord, and supporting classes with their attributes, methods, and associations

4.2.2 Sequence Diagrams

Sequence diagrams illustrate the interaction flow over time for key operations.

1. User Login Sequence

Figure 6 demonstrates the authentication process including credential validation, failed attempt tracking, and session management.

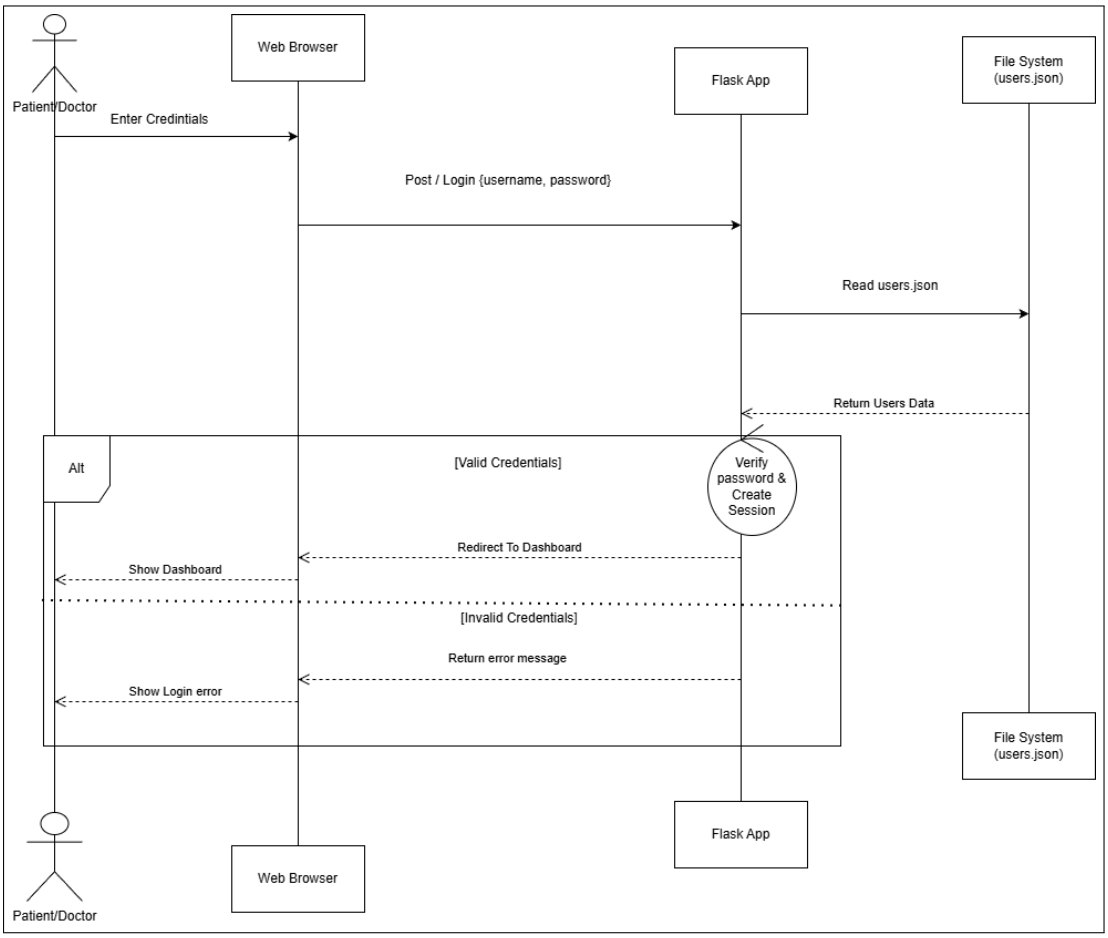


Figure 6: Login Sequence Diagram

2. Appointment Booking Sequence

Figure 7 shows the complete appointment booking workflow from availability checking to notification delivery.

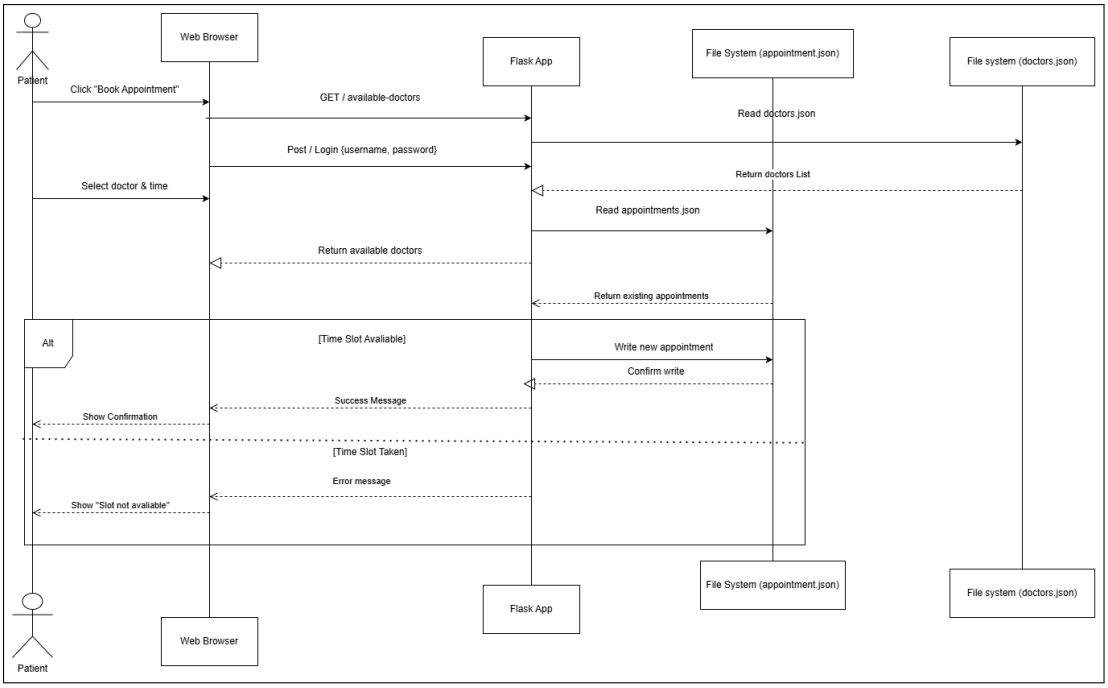


Figure 7: Appointment Booking Sequence Diagram

3. Medical Record Management Sequence

Figure 8 illustrates how doctors create medical records after consultations.

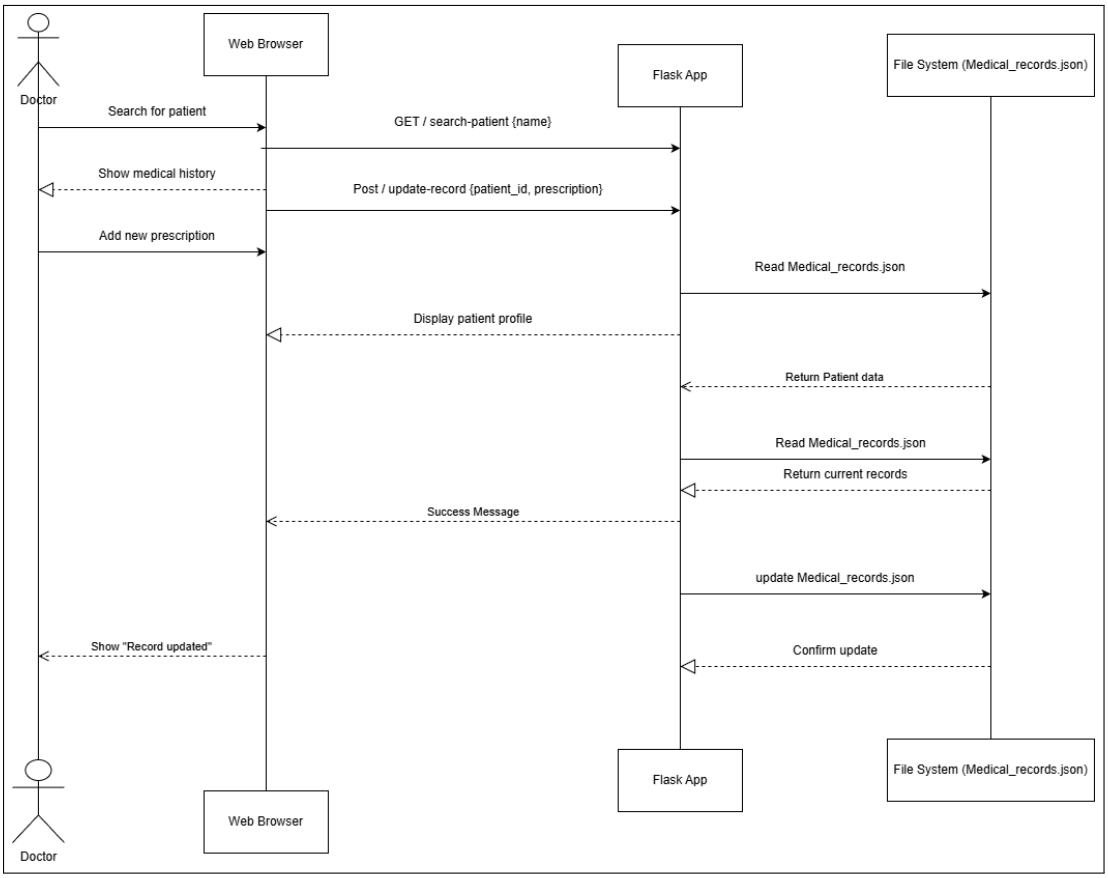


Figure 8: Medical Record Management Sequence Diagram

4. Report Generation Sequence

Figure 9 depicts the report generation process with data collection and formatting.

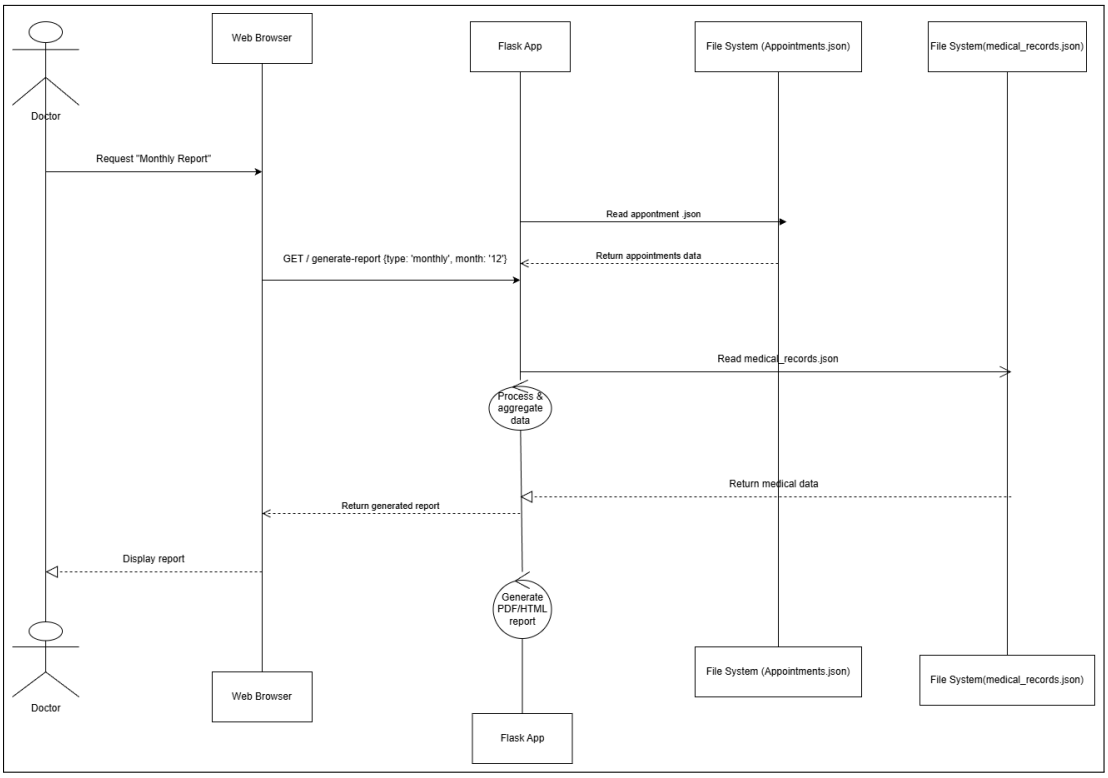


Figure 9: Report Generation Sequence Diagram

5. Search and Filter Sequence

Figure 10 shows the search functionality with filtering and pagination.

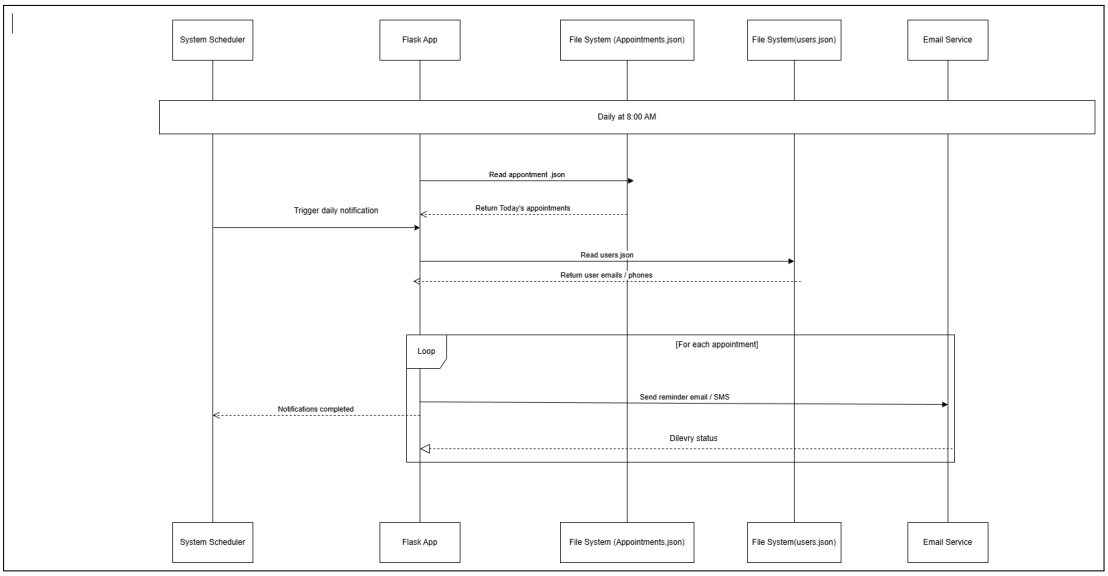


Figure 10: Search and Filter Sequence Diagram

4.3 UI/UX Design

4.3.1 Wireframes / Mockups

The following wireframes illustrate the layout and functionality of major user interfaces.

1. Login Page

Figure 11 presents the authentication interface with credential input and error handling.

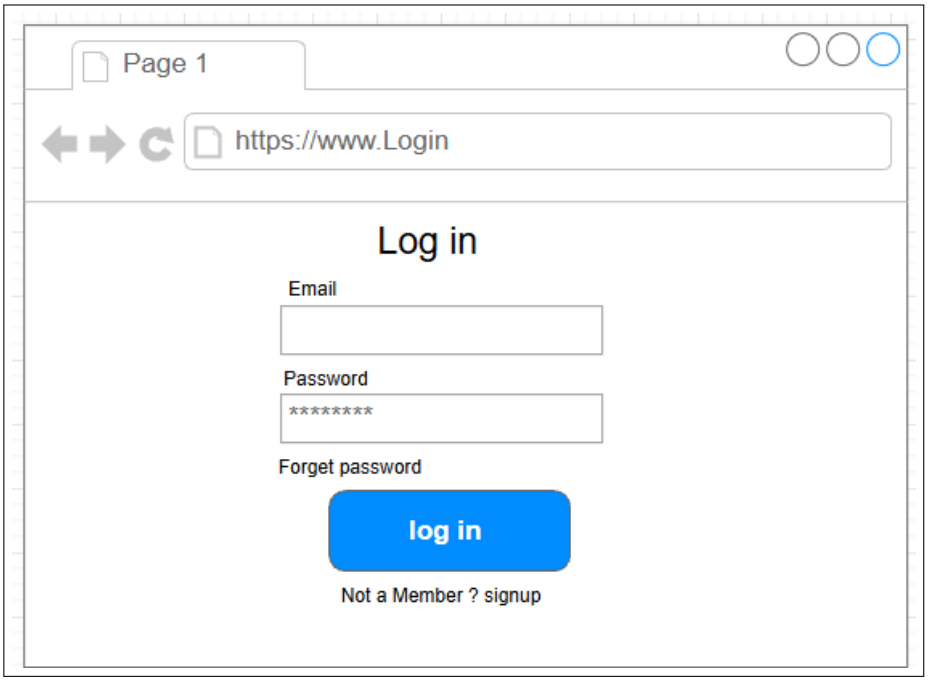


Figure 11: Login Page Wireframe

2. Patient Dashboard

Figure 12 shows the patient's main interface with appointments and quick actions.

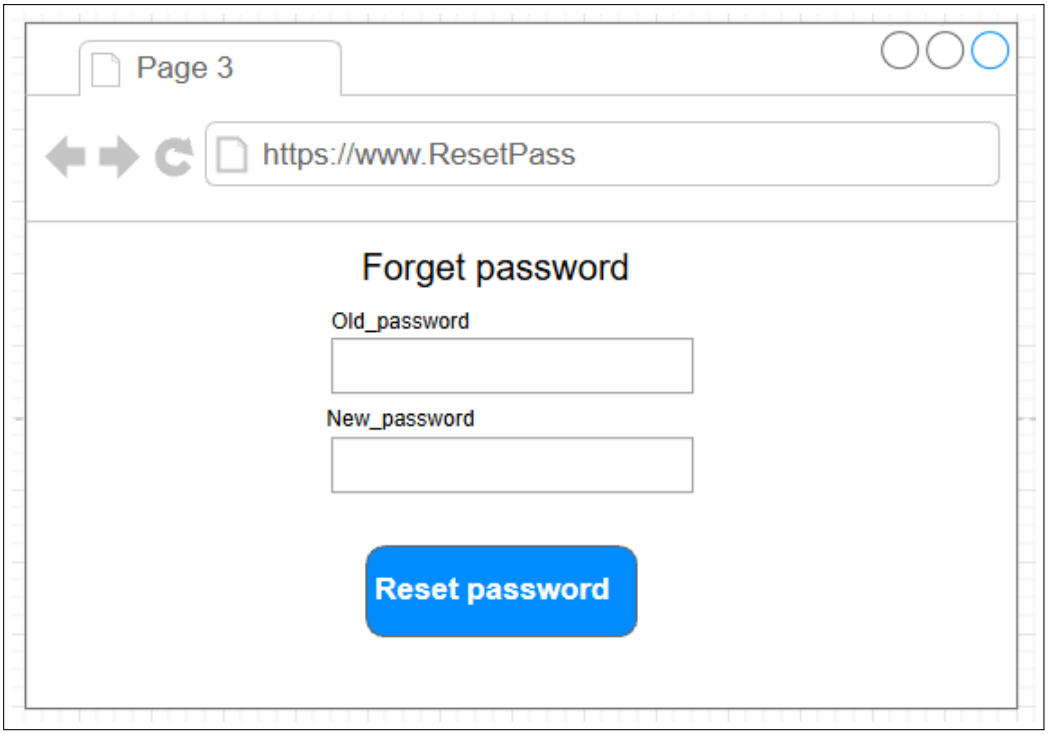


Figure 12: Patient Dashboard Wireframe

3. Doctor Dashboard

Figure 13 displays the doctor's schedule and patient management interface.

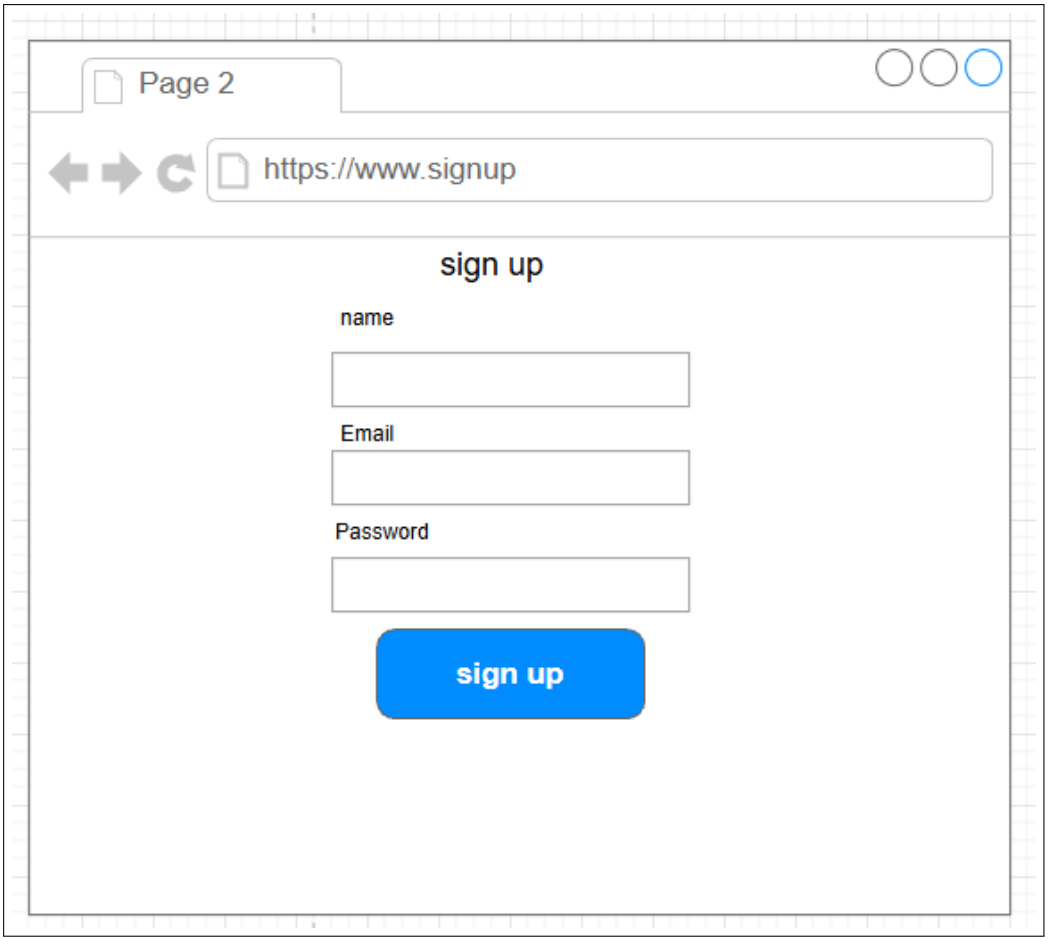


Figure 13: Doctor Dashboard Wireframe

4. Appointment Booking Page

Figure 14 illustrates the appointment booking workflow with doctor search and availability selection.

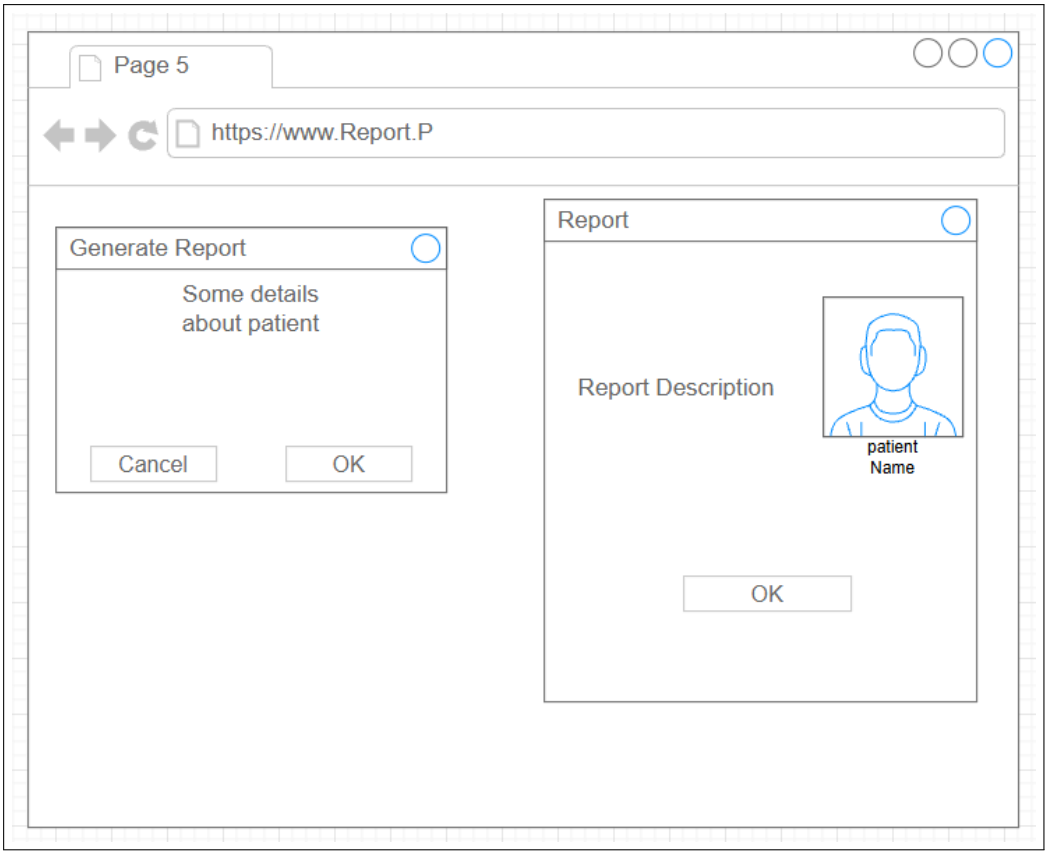


Figure 14: Appointment Booking Wireframe

5. Medical Record View

Figure 15 shows the comprehensive medical history display.

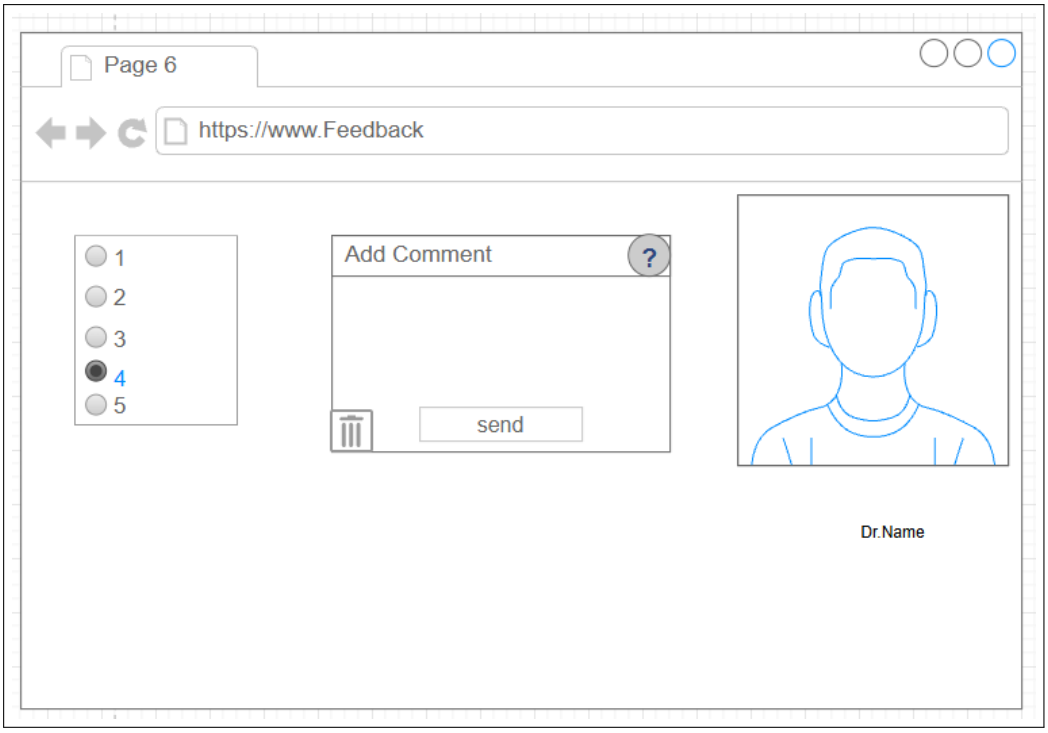


Figure 15: Medical Record Wireframe

6. Profile Management Page

Figure 16 presents the user profile management interface.

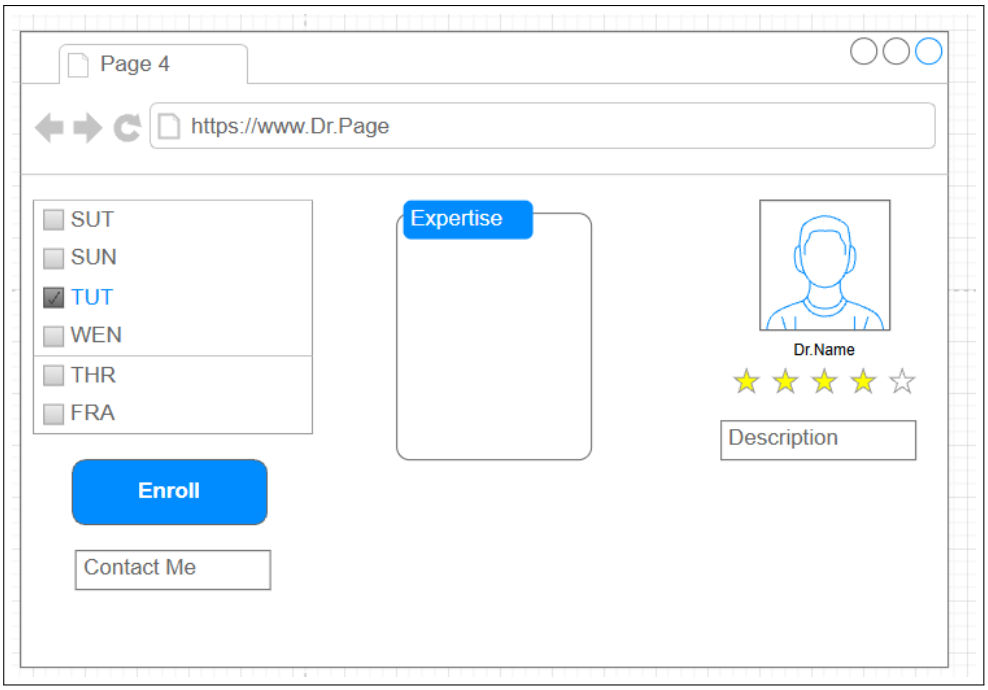


Figure 16: Profile Management Wireframe

4.4 Data Design

4.4.1 File Structure / Data Storage Model

The Clinexa system uses nine text files for data storage in the `clinexa_data/` directory:

```
clinexa_data/
 users.txt
 patients.txt
 doctors.txt
 appointments.txt
 medical_records.txt
 notifications.txt
 authentication_logs.txt
 feedback.txt
 reports.txt
```

Data Format Conventions:

- **Primary Separator:** Pipe (—) separates fields
- **List Separator:** Semicolon (;) for multiple items
- **Key-Value Separator:** Tilde (˜) for structured data
- **Encoding:** UTF-8
- **Null Values:** "NULL" or "None"

4.4.2 File Schemas

1. users.txt

```
userID|username|password|name|email|phone|role|created_date
```

Example:

```
1|ahmed_ragab|hashed_pass123|Ahmed Ragab|ahmed@example.com
   |+201234567890|patient|2025-01-15
```

2. patients.txt

```
patientID|userID|gender|date_of_birth|blood_type|allergies|
    medical_history|emergency_contact
```

Example:

```
1|1|male|1995-05-20|O+|Penicillin;Peanuts|Diabetes Type 2;
    Hypertension|Fatma Ragab:+201234567891
```

3. doctors.txt

```
doctorID|userID|specialization|license_number|consultation_fee|
    availability_days|availability_hours|years_experience
```

Example:

```
1|2|Cardiology|LIC123456|500|Sunday;Monday;Wednesday;Thursday
    |09:00-17:00|15
```

4. appointments.txt

```
appointmentID|patientID|doctorID|appointment_date|
    appointment_time|status|reason|meeting_link|created_date|
    updated_date
```

Example:

```
1|1|1|2025-02-15|10:00|confirmed|Regular checkup|https://meet.
    clinexa.com/apt001|2025-01-20|2025-01-21
```

5. medical_records.txt

```
recordID|appointmentID|patientID|doctorID|diagnosis|prescription|
    notes|vital_signs|record_date|record_time
```

Example:

```
1|1|1|1|Type 2 Diabetes Monitoring|Metformin 500mg~2 times daily;
    Insulin~10 units|Patient improving|BP~120/80;HR~72;Temp
    ~36.8|2025-02-15|10:30
```

6. notifications.txt

```
notificationID|recipientID|recipient_type|notification_type|
   message|sent_date|sent_time|status|channel
```

Example:

```
1|1|patient|appointment_reminder|Your appointment tomorrow at
   10:00 AM|2025-02-14|09:00|sent|email
```

7. authentication_logs.txt

```
logID|userID|username|attempt_date|attempt_time|status|ip_address
   |failed_attempts_count|is_blocked|block_until
```

Example:

```
1|1|ahmed_ragab|2025-01-20|14:30:25|success|192.168.1.100|0|false
   |NULL
```

8. feedback.txt

```
feedbackID|patientID|doctorID|appointmentID|rating|comment|
   feedback_date|feedback_time|status
```

Example:

```
1|1|1|1|5|Dr. Smith was very professional.|2025-02-15|15:45|
   submitted
```

9. reports.txt

```
reportID|report_type|generated_by|parameters|generated_date|
   generated_time|file_path
```

Example:

```
1|daily_appointments|2|date~2025-02-15;doctor~all
   |2025-02-15|17:00|/reports/daily_2025-02-15.pdf
```

5 Conclusion

5.1 Summary of Design Decisions

This Software Design Document has presented a comprehensive design for the Clinexa Medical System. Key design decisions include:

- Three-tier monolithic architecture with clear layer separation
- MVC design pattern for all nine system components
- File-based storage with structured text files
- Role-specific user interfaces with intuitive navigation
- Robust security through password hashing and access control

5.2 Design Phase Outputs

The design phase has produced:

- Architectural diagrams showing three-tier design
- Complete MVC component mapping
- Class diagram with all system classes
- Five sequence diagrams for key operations
- Six UI wireframes for major interfaces
- Complete file schemas for all nine data files

5.3 Alignment with Requirements

The design fully addresses all functional requirements (authentication, appointments, medical records, notifications, reports, search, feedback) and non-functional requirements (usability, reliability, security, maintainability, compatibility).

5.4 Next Steps

1. Review and approval from stakeholders
2. Development environment setup
3. Implementation planning with task assignment
4. Iterative development starting with core functionality
5. Comprehensive testing throughout implementation